

Aerospace Models

F16

Linear Longitudinal

Nonlinear Longitudinal

Nonlinear 6-DoF Angular

Aircraft Damage Modeling

B737

Nonlinear 6-DoF

B747

Classic State-Space

Normalized State-Space

Nonlinear 6-DoF

Usage examples

ELV

Typical Rocket

ComSat

F4C

GeoSat

UAV

Quadrotor

Nonlinear 6-DoF

Usage examples

Skywalker X8 (small UAV)

Nonlinear 6-DoF

AAI RQ-7 Shadow (class-II UAV)

Nonlinear 6-DoF

X15

Classic State-Space

Normalized State-Space

Nonlinear 6-DoF (Hypersonic)

LSU

Classic State-Space

Normalized State-Space

SUAV

Classic State-Space

Normalized State-Space

ComSat Improved

ELV Improved

F4C Improved

Missile Improved

Unity Environment

McDonnell Douglas F-4C — Longitudinal Dynamics

The F-4C Phantom II is an American 3rd-generation fighter bomber. This page mirrors the ELV layout: quick start, math model, derivatives, and API.



 **Quick start**

Launch the environment or the model within minutes.

[→ See example](#)

 **Model API**

Python class documentation for the F-4C longitudinal dynamics.

[→ Go to API](#)

 **Gymnasium environment**

Ready environment for RL agents.

[→ Explore](#)

 **Theory**

State equations and numerical parameters.

[→ Learn more](#)

Control object structure

The model is defined in the state space:

$$\dot{x} = Ax + Bu, \quad y = Cx + Du$$

where:

$$x = \begin{bmatrix} u & w & q & \theta \end{bmatrix}^\top, \quad u_{in} = \delta_e$$

The typical matrix structure is:

$$\begin{bmatrix} \dot{u} \\ \dot{w} \\ \dot{q} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} x_u & x_w & x_q & x_\theta \\ z_u & z_w & z_q & z_\theta \\ m_u & m_w & m_q & m_\theta \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u \\ w \\ q \\ \theta \end{bmatrix} + \begin{bmatrix} x_{\delta_e} \\ z_{\delta_e} \\ m_{\delta_e} \\ 0 \end{bmatrix} \delta_e$$

Variables	Coefficients
u: longitudinal speed, m/s w: vertical speed, m/s q: pitch rate, rad/s θ: pitch angle, rad δ_e: elevator deflection, rad	
 Units Angles and angular rates are in radians. API methods can return values in degrees.	

Mathematical model

$$\dot{x} = Ax + Bu, \quad y = Cx + Du$$

The model is described by the standard state-space equation:

$$\dot{x} = Ax + Bu$$

where:

- x** – state vector, $x = \begin{bmatrix} u, w, q, \theta \end{bmatrix}^\top$, representing deviations of longitudinal velocity, vertical velocity, pitch rate, and pitch angle.
- u** – control vector, in this case $u = \begin{bmatrix} \delta_e \end{bmatrix}$, where δ_e is the elevator deflection.
- A** – state matrix (or system matrix).
- B** – control matrix.

Units

State vector $x = \begin{bmatrix} u, w, q, \theta \end{bmatrix}^\top$:

- u, w**: m/s (velocities)
- q**: rad/s (angular velocity)
- θ**: rad (angle)

Control vector $u = \begin{bmatrix} \delta_e \end{bmatrix}$:

- δ_e**: rad (elevator deflection)

Flight Conditions

The computed matrices **A** and **B** for the F-4C are provided for the following flight conditions:

- Mach number**: 0.6
- Altitude**: 35,000 feet

Numerical Matrices

$$\begin{bmatrix} \dot{u} \\ \dot{w} \\ \dot{q} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 7.6180 \times 10^{-4} & 4.7612 \times 10^{-3} & 0 & -9.8100 \\ -6.6657 \times 10^{-2} & -2.8567 \times 10^{-1} & 1.8000 \times 10^2 & 0 \\ 1.5124 \times 10^{-3} & 1.0092 \times 10^{-2} & 1.6284 \times 10^{-1} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ w \\ q \\ \theta \end{bmatrix} +$$

Table of contents

Control object structure

Mathematical model

Units

Flight Conditions

Numerical Matrices

Derivatives (numerical values)

Sources

Reward

Quick start

Python API

LongitudinalF4C

import_linear_system

initialise_system

run_step

update_system_attributes

get_state

get_control

get_output

plot_output

LinearLongitudinalF4C

reward

step

reset

render

$$\begin{bmatrix} \ddot{u} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 1.5124 \times 10^{-3} & -1.0083 \times 10^{-2} & -1.6384 \times 10^{-1} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u \\ \theta \end{bmatrix}$$

Derivatives (numerical values)

- Matrix A (derivatives):

Coefficient	Value
x_u	7.6180×10^{-4}
x_w	4.7612×10^{-3}
x_q	0.0
x_θ	-9.8100
z_u	-6.6657×10^{-2}
z_w	-2.8567×10^{-1}
z_q	1.8000×10^2
z_θ	0
m_u	1.5124×10^{-3}
m_w	-1.0083×10^{-2}
m_q	-1.6384×10^{-1}
m_θ	0

- Input δe (column B):

Coefficient	Value
x_δe	2.6533×10^{-3}
z_δe	-6.8562
m_δe	-5.4446

 **Actuator limits**

The default control bounds are:

- Maximum magnitude: $\pm 25^\circ$
- Maximum rate: $\backslash(60^\circ\text{circ}/\text{text{s}\backslash)$

Internal computations use radians; the bounds are converted accordingly.

Sources

- Heffley R. K., Jewell W. F. Aircraft handling qualities data. – NASA, 1972. № AD-A277031.
- Etkin B., Reid L. D. Dynamics of flight. – New York : Wiley, 1959. – Vol. 2

Reward

The default reward function returns the negative absolute tracking error for the pitch angle:

$$r_t = -|\theta(t) - \theta_{\text{ref}}(t)|$$

Higher reward (closer to 0) indicates better tracking performance. A custom reward function can be passed via the `reward_func` parameter.

Quick start

Gymnasium

Model only

```
import gymnasium as gym
import numpy as np

from tensoraerospace.envs import LinearLongitudinalF4C
from tensoraerospace.utils import generate_time_period
from tensoraerospace.signals.standard import unit_step

dt = 0.01
tp = generate_time_period(tn=20, dt=dt)
number_time_steps = len(tp)
reference_signals = unit_step(degree=5, tp=tp, time_step=10, output_rad=True).reshape

env = gym.make(
    'LinearLongitudinalF4C-v0',
    number_time_steps=number_time_steps,
    initial_state=[[0],[0],[0],[0]],
    reference_signal=reference_signals,
)
state, info = env.reset()
for _ in range(200):
    action = np.array([[0.1]])
    state, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        break
```

Python API

Model

Gymnasium environment

LongitudinalF4C(x0, number_time_steps,

selected_state_output=None, t0=0, dt=0.01)

Bases: `ModelBase`

McDonnell Douglas F-4C in longitudinal control channel.

Parameters:

Name	Type	Description	Default
<code>x0</code>	<code>ndarray list[float]</code>	Initial state of the control object.	<i>required</i>
<code>number_time_steps</code>	<code>int</code>	Number of time steps.	<i>required</i>
<code>selected_state_output</code>	<code>optional</code>	Selected states of the control object. Defaults to None.	<code>None</code>
<code>t0</code>	<code>int</code>	Initial time. Defaults to 0.	<code>0</code>
<code>dt</code>	<code>float</code>	Discretization frequency. Defaults to 0.01.	<code>0.01</code>

Action space

ele: elevator [rad]

State space

u: Longitudinal aircraft velocity [ft/s] w: Normal aircraft velocity [ft/s] q: Pitch angular velocity [rad/s] theta: Pitch angle [rad]

Output space

u: Longitudinal aircraft velocity [ft/s] w: Normal aircraft velocity [ft/s] q: Pitch angular velocity [rad/s] theta: Pitch angle [rad]

Initialize LongitudinalF4C instance.

Parameters:

Name	Type	Description	Default
<code>x0</code>	<code>ndarray list[float]</code>	Initial state of the control object.	<i>required</i>
<code>number_time_steps</code>	<code>int</code>	Number of time steps.	<i>required</i>
<code>selected_state_output</code>	<code>list[int] None</code>	Selected states of the control object. Defaults to None.	<code>None</code>
<code>t0</code>	<code>float</code>	Initial time. Defaults to 0.	<code>0</code>
<code>dt</code>	<code>float</code>	Discretization frequency. Defaults to 0.01.	<code>0.01</code>

```
import_linear_system()
```

Load (set) stored linearized system matrices.

```
initialise_system(x0, number_time_steps)
```

Initialize the system and allocate history buffers.

Parameters:

Name	Type	Description	Default
<code>x0</code>	<code>ndarray list[float]</code>	Initial state.	<i>required</i>
<code>number_time_steps</code>	<code>int</code>	Number of simulation steps.	<i>required</i>

```
run_step(ut_0)
```

Run one discrete-time simulation step.

Parameters:

Name	Type	Description	Default
<code>ut_0</code>	<code>ndarray</code>	Control vector.	<i>required</i>

Returns:

Type	Description
<code>ndarray</code>	<code>np.ndarray</code> : Next state at time t+1.

```
update_system_attributes()
```

Update time-dependent attributes after each simulation step.

```
get_state(state_name, to_deg=False, to_rad=False)
```

Return the time history of a state.

Parameters:

Name	Type	Description	Default
<code>state_name</code>	<code>str</code>	State name.	<i>required</i>
<code>to_deg</code>	<code>bool</code>	Convert radians to degrees.	<code>False</code>
<code>to_rad</code>	<code>bool</code>	Convert degrees to radians.	<code>False</code>

Returns:

Type	Description
<code>ndarray</code>	<code>np.ndarray</code> : State history array.

`get_control(control_name, to_deg=False, to_rad=False)`

Return the time history of a control input.

Parameters:

Name	Type	Description	Default
<code>control_name</code>	<code>str</code>	Control signal name.	<i>required</i>
<code>to_deg</code>	<code>bool</code>	Convert radians to degrees.	<code>False</code>
<code>to_rad</code>	<code>bool</code>	Convert degrees to radians.	<code>False</code>

Returns:

Type	Description
<code>ndarray</code>	<code>np.ndarray</code> : Control history array.

`get_output(state_name, to_deg=False, to_rad=False)`

Return the time history of an output signal.

Parameters:

Name	Type	Description	Default
<code>state_name</code>	<code>str</code>	Output name.	<i>required</i>
<code>to_deg</code>	<code>bool</code>	Convert radians to degrees.	<code>False</code>
<code>to_rad</code>	<code>bool</code>	Convert degrees to radians.	<code>False</code>

Returns:

Type	Description
<code>ndarray</code>	<code>np.ndarray</code> : Output history array.

`plot_output(output_name, time, lang='rus', to_deg=False, to_rad=False, figsize=(10, 10))`

Plot an output signal over time.

Parameters:

Name	Type	Description	Default
<code>output_name</code>	<code>str</code>	Output name.	<i>required</i>
<code>time</code>	<code>ndarray</code>	Time vector.	<i>required</i>
<code>lang</code>	<code>str</code>	Axis label language ('rus' or 'eng'). Defaults to 'rus'.	<code>'rus'</code>
<code>to_deg</code>	<code>bool</code>	Convert radians to degrees.	<code>False</code>
<code>to_rad</code>	<code>bool</code>	Convert degrees to radians.	<code>False</code>
<code>figsize</code>	<code>tuple</code>	Figure size.	<code>(10, 10)</code>

Returns:

Type	Description
<code>Figure</code>	<code>matplotlib.figure.Figure</code> : Figure object.